

Neovim Markdown Shortcuts

Reference

Full command reference for the full-feature Neovim markdown setup

Setup Instructions

Linux

1. Install Neovim (0.10+; this config targets 0.11+/0.12), plus the tools plugins need to build things:

```
sudo apt install neovim git pandoc wkhtmltopdf nodejs npm build-essential
```

(Debian/Ubuntu; use dnf/pacman etc. on other distros. build-essential supplies gcc, needed to compile Treesitter parsers.)

2. Install the Tree-sitter CLI (nvim-treesitter's main branch shells out to it to compile parsers):

```
npm install -g tree-sitter-cli
```

3. Run the bundled setup script from inside this folder — it checks the dependencies above, offers to install the Tree-sitter CLI if missing, and symlinks this project to ~/.config/nvim (backing up any existing config first, with confirmation):

```
./setup.sh
```

Or do it by hand instead: back up anything existing, then symlink this whole directory as your Neovim config folder:

```
mv ~/.config/nvim ~/.config/nvim.bak # back up anything existing first
ln -s /home/ed/MEGA/app/vim_for_markdown/02_fullfeature_nvim ~/.config/nvim
```

or run Neovim with an isolated config so it doesn't touch your normal setup:

```
NVIM_APPNAME=nvim-markdown nvim
```

after symlinking/copying this folder to ~/.config/nvim-markdown instead.

4. Launch Neovim once and wait. On first run, lazy.nvim clones every plugin and mason.nvim installs the language servers (html, cssls, pyright, marksman) automatically — this needs network access and takes a minute or two:

```
nvim
```

Run `:Lazy` to watch plugin install progress, or `:Mason` to watch language server install progress.

5. Open a markdown file to confirm it all works:

```
nvim notes.md
```

Windows

1. Install Neovim, Git, Pandoc, Node.js, and a C compiler:

```
winget install Neovim.Neovim
winget install Git.Git
winget install JohnMacFarlane.Pandoc
winget install wkhtmltopdf.wkhtmltopdf
winget install OpenJS.NodeJS
winget install --id BurntSushi.ripgrep.MSVN
```

For the C compiler needed to build Treesitter parsers, install the [Visual Studio Build Tools](#) (“Desktop development with C++” workload), or use the compiler bundled with [MSYS2/MinGW](#) instead.

2. Install the Tree-sitter CLI:

```
npm install -g tree-sitter-cli
```

3. Find/create your Neovim config folder — normally:

```
$env:LOCALAPPDATA\nvim
```

Copy (or symlink, from an elevated PowerShell) this project’s folder there:

```
New-Item -ItemType SymbolicLink -Path "$env:LOCALAPPDATA\nvim" -
Target "C:\path\to\vim_for_markdown\02_fullfeature_nvim"
```

4. Launch Neovim once and wait for `lazy.nvim` and `mason.nvim` to finish installing plugins/language servers (needs network access):

```
nvim
```

5. Open a markdown file to confirm it all works:

```
nvim notes.md
```

The bundled preview/export commands use `xdg-open` (Linux) to launch the browser. On Windows, edit `lua/config/markdown-tools.lua` and replace `xdg-open` with `cmd /c start` so previews open in your default browser.

What This Setup Gives You

The leader key in this config is Space (not `\` as in plain Vim). Once set up, opening a `.md` file gives you:

- **Light/dark colour theme toggle** — `<leader>d` switches between Catppuccin Latte (light) and Catppuccin Mocha (dark); starts in dark mode.
- **Real syntax highlighting via Treesitter** — markdown, embedded HTML/CSS/Python/YAML front matter are all parsed and highlighted accurately, not just pattern-matched.
- **Concealment / rendered markdown** — headings, bold/italic, bullets and horizontal rules are displayed formatted in-buffer instead of showing raw `**/_/#` symbols (via `render-markdown.nvim`).
- **Live-reloading browser preview** — `<leader>p` opens a scroll-synced live preview in your browser that updates as you type.
- **Static HTML/PDF export** — `<leader>eh` exports to a standalone HTML file and opens it; `<leader>ep` exports straight to PDF (both via Pandoc).
- **Table auto-formatting** — toggle with `<leader>tm`; markdown tables realign their | columns as you edit.
- **Auto-continuing lists** — pressing Enter inside a bullet or numbered list automatically inserts the next marker.
- **Folding by heading** — collapse/expand sections with `za/zo/zc` (folds start open).
- **Distraction-free writing mode** — `<leader>z` toggles a centered, chrome-free writing view.
- **Word count / reading time** — shown live in the statusline whenever editing a markdown file.
- **Table of contents generation** — `<leader>tc` inserts/updates a ToC built from your headings.
- **Internal link checking** — `<leader>lc` reports any `[text](#anchor)` link that doesn't match a heading in the file; `gf` jumps to another markdown file referenced by a relative link.
- **Snippets** — common markdown/HTML/Python boilerplate (tables, code fences, front matter, etc.) available via completion while typing.
- **LSP-powered autocomplete** — real autocomplete, hover docs (K), go-to-definition (gd) for HTML, CSS and Python files, plus a markdown language server for heading/link awareness.
- **Git integration** — added/changed/removed line markers in the gutter, wherever your markdown notes live in a git repo.
- **Session persistence** — `<leader>sl` restores your last session (open files, cursor position, window layout).
- **Fuzzy file finding** — `<leader>ff` to open any file by fuzzy name, `<leader>fr` for recently-opened files.
- **Quick “save as”** — `<leader>sa` opens a telescope-file-browser popup to browse to a folder, `Ctrl-s` to pick it, then type just the filename.
- **Filetype switching** — force the buffer to HTML/Python/CSS/Markdown highlighting and LSP with `<leader>fh` / `<leader>fp` / `<leader>fc` / `<leader>fm`.
- **All of standard Vim** — every normal Vim/Neovim motion, operator, and command below still works exactly as usual.

This Setup's Custom Shortcuts (leader = Space)

<code><leader>p</code>	Toggle live browser preview (markdown-preview.nvim)
<code><leader>eh</code>	Export to HTML and open in browser
<code><leader>ep</code>	Export to PDF
<code><leader>lc</code>	Check internal [text] (#anchor) links resolve
<code><leader>h</code>	Open this shortcut reference (nvim_shortcuts.md)
<code><leader>rt</code>	Force a full terminal repaint (fixes stray red blocks/redraw glitches some terminals leave behind)
<code><leader>tm</code>	Toggle table mode (auto-align tables)
<code><leader>tc</code>	Insert/update table of contents
<code><leader>z</code>	Toggle zen (distraction-free) mode
<code><leader>ts</code>	Toggle spell-check
<code><leader>tw</code>	Toggle the 80-column limit (textwidth + guide line) on/off
<code><leader>gq</code>	Reflow current paragraph to 80 columns (fixes existing/pasted text; textwidth only wraps as you actively type)
<code><leader>gG</code>	Reflow the whole file to 80 columns
<code><leader>d</code>	Toggle light/dark colour theme
<code><leader>sa</code>	Save as... — browse to a folder with telescope-file-browser, press <code>Ctrl-s</code> , then type the filename
<code><leader>w</code>	Save file
<code><leader>q</code>	Quit
<code><leader>fh</code>	Switch filetype to HTML
<code><leader>fp</code>	Switch filetype to Python
<code><leader>fc</code>	Switch filetype to CSS
<code><leader>fm</code>	Switch filetype back to Markdown
<code><leader>ff</code>	Find file (Telescope)
<code><leader>fg</code>	Grep across project (Telescope)
<code><leader>fb</code>	Find open buffer (Telescope)
<code><leader>fr</code>	Recently opened files (Telescope)
<code><leader>ss</code>	Restore a saved session
<code><leader>sl</code>	Restore last session
<code><leader>rn</code>	LSP: rename symbol
<code><leader>ca</code>	LSP: code action
<code>gd</code>	LSP: go to definition
<code>K</code>	LSP: hover documentation
<code>Ctrl-h/j/k/l</code>	Move focus between windows

Modes

i	Insert before cursor
I	Insert at start of line
a	Insert after cursor (append)
A	Insert at end of line
o	Open new line below and insert
O	Open new line above and insert
Esc	Return to Normal mode
v	Visual mode (character-wise)
V	Visual line mode
Ctrl-v	Visual block mode
R	Replace mode
gi	Insert at last insert position
:	Command-line mode

Cursor Movement (basic)

h j k l	Left, down, up, right
w	Next word start
W	Next WORD start (whitespace-separated)
e	End of word
E	End of WORD
b	Back to word start
B	Back to WORD start
0	Start of line
^	First non-blank character of line
\$	End of line
f{char}	Jump to next occurrence of char
F{char}	Jump to previous occurrence of char
t{char}	Jump till before next occurrence of char
;	Repeat last f/t/F/T
,	Repeat last f/t/F/T in opposite direction

Cursor Movement (lines & screen)

gg	Go to first line of file
G	Go to last line of file
{n}G	Go to line n
H	Top of screen
M	Middle of screen

L	Bottom of screen
Ctrl-d	Scroll half page down
Ctrl-u	Scroll half page up
zz	Center current line on screen
{	Previous paragraph/blank line
}	Next paragraph/blank line
%	Jump to matching bracket/paren
Ctrl-o	Jump to older cursor position
Ctrl-i	Jump to newer cursor position

Editing / Deleting

x	Delete character under cursor
dd	Delete (cut) current line
dw	Delete to next word start
d\$ / D	Delete to end of line
d{motion}	Delete over any motion (generic)
cc / S	Change (delete + insert) whole line
cw	Change to next word start
C	Change to end of line
r{char}	Replace single character
s	Delete character and insert
~	Toggle case of character under cursor
J	Join line below with current line
u	Undo
Ctrl-r	Redo
.	Repeat last change

Copy / Paste (Yank)

yy / Y	Yank (copy) current line
yw	Yank word
y{motion}	Yank over any motion (generic)
p	Paste after cursor / below line
P	Paste before cursor / above line
"{reg}y	Yank into named register
"{reg}p	Paste from named register

Visual Mode

<code>v</code>	Start character-wise visual selection
<code>V</code>	Start line-wise visual selection
<code>Ctrl-v</code>	Start block-wise visual selection
<code>gv</code>	Reselect last visual selection
<code>d / x</code>	Delete selection
<code>y</code>	Yank selection
<code>c</code>	Change selection
<code>> / <</code>	Indent / un-indent selection
<code>=</code>	Auto-indent selection
<code>J</code>	Join selected lines

Search & Replace

<code>/pattern</code>	Search forward
<code>?pattern</code>	Search backward
<code>n / N</code>	Repeat search, same/opposite direction
<code>* / #</code>	Search word under cursor forward/backward
<code>:noh</code>	Clear search highlighting
<code>:%s/old/new/g</code>	Replace all occurrences in file
<code>:%s/old/new/gc</code>	Replace all, with confirmation

Files & Buffers

<code>:w</code>	Write (save) file
<code>:w {file} / :saveas</code>	Save as
<code>:q</code>	Quit
<code>:wq / ZZ</code>	Save and quit
<code>:e {file}</code>	Edit (open) a file
<code>:bn / :bp</code>	Next / previous buffer
<code>:bd</code>	Delete (close) buffer
<code>:ls</code>	List open buffers
<code>gf</code>	Go to file under cursor (resolves markdown links)

Windows & Tabs

<code>:split / :vsplit</code>	Horizontal / vertical split
<code>Ctrl-w w</code>	Cycle to next window
<code>Ctrl-h/j/k/l</code>	Move to window left/down/up/right (mapped in this config)

Ctrl-w q	Close current window
Ctrl-w =	Equalize window sizes
:tabnew	Open new tab
gt / gT	Next / previous tab

Marks & Registers

m{a-z}	Set mark at cursor position
'{a-z} / `{a-z}	Jump to mark (line / exact position)
:marks	List all marks
:reg	List all registers
qa ... q	Record macro into register a, then stop
@a	Play macro from register a
@@	Repeat last played macro

Text Objects (used with d/c/y/v)

iw / aw	Inner word / a word
ip / ap	Inner paragraph / a paragraph
i" / a"	Inner / a quoted string
i(/ a(	Inner / a parentheses block
i{ / a{	Inner / a braces block
it / at	Inner / a tag block (HTML)

Example: di" deletes inside quotes, ca(changes a parenthesised block including parens.

Folding (Treesitter-based)

zf{motion}	Create fold over motion
zo / zc / za	Open / close / toggle fold
zR / zM	Open all folds / close all folds
zj / zk	Move to next / previous fold

Spell-check

on by default	Markdown files have spell-check enabled automatically
]s / [s	Jump to next / previous misspelled word
z=	Suggest corrections for word under cursor
zg	Add word to dictionary
zw	Mark word as wrong

Miscellaneous

Ctrl-a / Ctrl-x	Increment / decrement number under cursor
:{cmd}	Run external shell command
:checkhealth	Diagnose plugin/LSP/treesitter setup issues
:Lazy	Open the plugin manager UI
:Mason	Open the LSP server manager UI
:help {topic}	Open built-in help for topic

Neovim Shortcuts Reference — part of the Full-Feature Neovim Markdown Setup project.